

# Shor's Algorithm

# Overview

- **Shor's algorithm** is a quantum algorithm for factoring a number  $N$  in  $O((\log N)^3)$  time and  $O(\log N)$  space, named after Peter Shor.
- The algorithm is significant because it implies that RSA, a popular public-key cryptography method, might be easily broken, given a sufficiently large quantum computer

# Overview

- Like many quantum computer algorithms, Shor's algorithm is probabilistic
- It gives the correct answer with high probability, and the probability of failure can be decreased by repeating the algorithm.

# Overview

- Shor's algorithm was discovered in 1994 by Peter Shor, but the classical part was known before.
- It is credited to G. L. Miller. Seven years later, in 2001.
- It was demonstrated by a group at IBM, which factored 15 into 3 and 5, using a quantum computer with 7 qubits.



Peter  
Shor 1994

## Shor's Algorithm

- Shor's algorithm shows (in principle,) that a quantum computer is capable of factoring very large numbers in polynomial time.

The algorithm is dependant on

- Modular Arithmetic
- Quantum Parallelism
- Quantum Fourier Transform

## Shor's Algorithm - Periodicity

- An important result from Number Theory:

**$F(a) = x^a \bmod N$**  is a periodic function

- Choose  $N = 15$  and  $x = 7$  and we get the following:

$$7^0 \bmod 15 = 1$$

$$7^1 \bmod 15 = 7$$

$$7^2 \bmod 15 = 4$$

$$7^3 \bmod 15 = 13$$

$$7^4 \bmod 15 = 1$$

⋮

## Shor's Algorithm - In Depth Analysis

### **To Factor an odd integer $N$ (Let's choose 15) :**

1. Determine if the number  $n$  is a prime, a even number, or an integer power of a prime number. If it is we will not use Shor's algorithm. There are efficient classical methods for determining if a integer  $n$  belongs to one of the above groups, and providing factors for it if it is. This step would be performed on a classical computer.
2. Choose an integer  $q$  such that  $N^2 < q < 2N^2$  **let's pick 256**
3. Choose a random integer  $x$  such that  $\text{GCD}(x, N) = 1$  **let's pick 7**

## Shor's Algorithm - In Depth Analysis

4. Create two quantum registers (these registers must also be entangled so that the collapse of the input register corresponds to the collapse of the output register)
  - ❑ Input register: must contain enough qubits to represent numbers as large as  $q-1$ . up to 255, so we need 8 qubits
  - ❑ Output register: must contain enough qubits to represent numbers as large as  $N-1$ . up to 14, so we need 4 qubits

## Shor's Algorithm - Preparing Data

5. Load the input register with an equally weighted superposition of all integers from 0 to  $q-1$ . **0 to 255**
6. Load the output register with all zeros.

**The total state of the system at this point will be:**

$$\frac{1}{\sqrt{256}} \sum_{a=0}^{255} |a, 0000\rangle$$

Input Register      Output Register

Note: the comma here denotes that the registers are entangled

## Shor's Algorithm - Modular Arithmetic

7. Apply the transformation  $x^a \bmod N$  to each number in the input register, storing the result of each computation in the output register.

Note that we are using decimal numbers here only for simplicity.

| Input Register | $7^a \bmod 15$ | Output Register |
|----------------|----------------|-----------------|
| $ 0\rangle$    | $7^0 \bmod 15$ | 1               |
| $ 1\rangle$    | $7^1 \bmod 15$ | 7               |
| $ 2\rangle$    | $7^2 \bmod 15$ | 4               |
| $ 3\rangle$    | $7^3 \bmod 15$ | 13              |
| $ 4\rangle$    | $7^4 \bmod 15$ | 1               |
| $ 5\rangle$    | $7^5 \bmod 15$ | 7               |
| $ 6\rangle$    | $7^6 \bmod 15$ | 4               |
| $ 7\rangle$    | $7^7 \bmod 15$ | 13              |

•  
•

## Shor's Algorithm - Superposition Collapse

8. Now take a measurement on the output register. This will collapse the superposition to represent *just one* of the results of the transformation, let's call this value  $c$ .

Our output register will collapse to represent one of the following:

$|1\rangle$ ,  $|4\rangle$ ,  $|7\rangle$ , or  $|13\rangle$

For sake of example, let's choose  $|1\rangle$

## Shor's Algorithm - Entanglement

*Now things really get interesting !*

9. Since the two registers are entangled, measuring the output register will have the effect of partially collapsing the input register into an **equal superposition** of each state between 0 and  $q-1$  that yielded  $c$  (the value of the collapsed output register.)

Since the output register collapsed to  $|1\rangle$ , the input register will partially collapse to:

$$\frac{1}{\sqrt{64}} |0\rangle + \frac{1}{\sqrt{64}} |4\rangle + \frac{1}{\sqrt{64}} |8\rangle + \frac{1}{\sqrt{64}} |12\rangle, \dots$$

The probabilities in this case are  $\frac{1}{\sqrt{64}}$  since our register is now in an equal superposition of 64 values (0, 4, 8, ... 252)

## Shor's Algorithm - QFT

We now apply the Quantum Fourier transform on the partially collapsed input register. The fourier transform has the effect of taking a state  $|a\rangle$  and transforming it into a state given by:

$$\frac{1}{\sqrt{q}} \sum_{c=0}^{q-1} |c\rangle * e^{2\pi i ac / q}$$

## Shor's Algorithm - QFT

$$\frac{1}{\sqrt{64}} \sum_{a \in A} |a\rangle, |1\rangle \longrightarrow \frac{1}{\sqrt{256}} \sum_{c=0}^{255} |c\rangle * e^{2\pi i a c / 256}$$

**Note:** A is the set of all values that  $7^a \bmod 15$  yielded 1.  
In our case  $A = \{0, 4, 8, \dots, 252\}$

So the final state of the input register after the QFT is:

$$\frac{1}{\sqrt{64}} \sum_{a \in A} \frac{1}{\sqrt{256}} \sum_{c=0}^{255} |c\rangle * e^{2\pi i a c / 256}, |1\rangle$$

## Shor's Algorithm - QFT

The QFT will essentially peak the probability amplitudes at integer multiples of  $q/r$ , where  $r$  is the desired period in our case  $r$  is 4.

$$|0\rangle, |64\rangle, |128\rangle, |192\rangle, \dots$$

So we no longer have an equal superposition of states, the probability amplitudes of the above states are now higher than the other states in our register.

Measure the state of register one, call this value  $m$ , this integer  $m$  has a very high probability of being a multiple of  $q/r$

With our knowledge of  $q$ , and  $m$ , there are methods of calculating the period (one method is the continuous fraction expansion of the ratio between  $q$  and  $m$ .)

## Shor's Algorithm - The Factors

10. Now that we have the period, the factors of  $N$  can be determined by taking the greatest common divisor of  $N$  with respect to  $x^{(P/2)} + 1$  and  $x^{(P/2)} - 1$ . The idea here is that this computation will be done on a classical computer.

We compute:

$$\text{Gcd}(7^{4/2} + 1, 15) = 5$$

$$\text{Gcd}(7^{4/2} - 1, 15) = 3$$

**We have successfully factored 15!**

What to do if Shor's algorithm failed to produce factors of  $n$

- The QFT comes up short and reveals the wrong period. This probability is actually dependant on your choice of  $q$ . The larger the  $q$ , the higher the probability of finding the correct probability.
- The period of the series ends up being odd
- Fourier transform could be measured to be 0, making the post processing in the next step impossible.
- The algorithm will sometimes find factors of 1 and  $n$ , which is not useful either.

If either of these cases occur, we go back to the beginning and pick a new  $x$ .

# References

- Peter W. Shor “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer “, SIAM Journal on Computing (1997)
- [www.eecis.udel.edu/~saunders/courses/879-03s/](http://www.eecis.udel.edu/~saunders/courses/879-03s/)
- [http://en.wikipedia.org/wiki/Shor's\\_algorithm](http://en.wikipedia.org/wiki/Shor's_algorithm)